Decoding Manchester Coded Transmissions in Hardware

Zachary Kohnen
Electrical Engineering
Eindhoven University of Technology
Eindhoven, Netherlands
z.j.kohnen@student.tue.nl

***Abstract*—Application specific integrated circuits (ASICs) are the logical next step from a field-programmable gate array (FPGA) design. However, development costs for ASICs have been a limiting factor in their accessability. This paper explores the steps required to design and test a manchester code receiver in verilog, producing a working low-cost ASIC through the use of Tiny Tapeout.**

I. Introduction

Hardware design, both for application specific integrated circuits (ASICs) and field-programmable gate arrays (FPGAs), is important to be able to realize modern digital designs. The small and compact silicon chip size allows for higher bandwidths than would be possible with discrete components, as well as greater design flexibility.

When lowering a FPGA design to silicon, there are 3 major roadblocks: Access to a process design kit (PDK), Access to IP blocks and libraries, Access to an electronics design automation suite or EDA. Getting access to these 3 components has always been challenging. There have been relatively small home-brew efforts to produce ASIC silicon fully free and open however they have not gained large support. Google has been focusing on developing the required connections and tools in order to create and foster a truly free and open source silicon design system. [1]

In the development of this open source silicon design system a collaboration between Google and Skywater Technologies has produced the fully open source SkyWater SKY130 PDK. [2] Along with open source tools such as the OSS Cad Suite compiled by YosysHQ [3] it has now become very possible for designers to produce manufacturable silicon designs. Although designers are able to create silicon designs, there remains a large cost barrier.

From their website, "Tiny Tapeout is an educational project that makes it easier and cheaper than ever to get your designs manufactured on a real chip!"[4]. Tiny Tapeout provides a lower cost to entry for silicon fabrication by submitting a silicon design made up of many smaller designs enclosed in a Caraval [5] SoC harness. These designs – or tiles – placed in a grid and multiplexed so that each design can individually take control of the 26 GPIO pins. Fig. 1 shows the layout of Tiny Tapeout's third run.

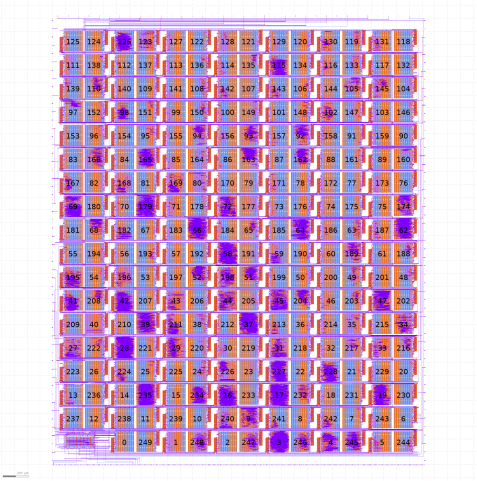


Fig. 1: Tiny Tapeout 3 die render with 249 separate projects [6]

Tiny Tapeout, in order to provide as seamless of a design experience as possible, provides automation tools to build and test users' hardware designs using OpenLane [7], cocotb, and Icarus Verilog on the SKY130 PDK. This automation allows both Tiny Tapeout and the designers to receive quick, standardized feedback of both the functionality, and the validity of the design.

This paper aims to explore the workflow provided by Tiny Tapeout, develop a manchester coding receiver, and assess the challenges and viability for others to implement such a receiver as part of a course program.

The paper is organized into 5 sections: the introduction, the system overview — where the transmission, its coding and format will be discussed, along with the tools used for reception and testing, the results — where the implementation details and experience with Tiny Tapeout is provided, and the discussion — covering problems with compatibility in the receiver

implementation, challenges faced when hardening, and ASIC optimizations that were implemented, and conclusions.

II. System Overview

The system consists of 4 components: a thermostat transmitter, a low-cost receiver frontend, a custom ASIC implementation, and a RP2040 microcontroller. The connections between these components can be seen in Fig. 2.

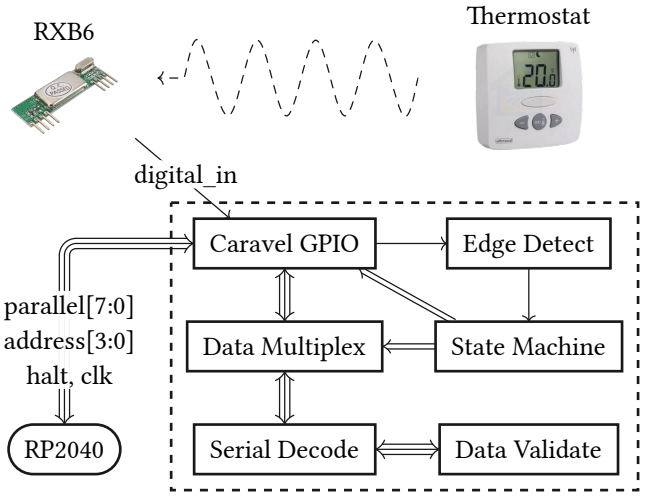


Fig. 2: System diagram

A. Transmission

The transmitter used in this project is a 433MHz wireless thermostat, specifically the "albrand 38005 Thermostat Basic LCD AAA RF433MHz". This thermostat transmits information to a control system attached to a furnace or air conditioning unit. There are no public documents that could be found documenting the transmission format or protocol used by these devices.

The first part of the project involved decoding and deciphering the messages transmitted from the thermostat. Previous work on the decoding of the transmissions found the thermostat to use on-off keying (OOK) — a special case of amplitude-shift keying (ASK).

a) *Manchester coding:* The transmissions are encoded using manchester encoding. Manchester coding makes use of two symbols, a low to high signal and a high to low signal as can be seen in Fig. 3. Due to the use of OOK the received signals are unipolar. Following the Gram-Schmidt Orthogonalization Procedure [8] there is determined to be 2 building block waveforms $\varphi_{\{1,2\}}[n]$ as signals $s_1[n]$ and $s_2[n]$ are orthogonal. Had the signaling scheme been chosen to be bipolar, the two signals for manchester coding would not have been orthogonal, resulting in only 1 building block waveform. However, a bipolar signaling scheme at pass-band requires the knowledge of phase at the receiver, dramatically increasing the complexity and cost of the receiver.

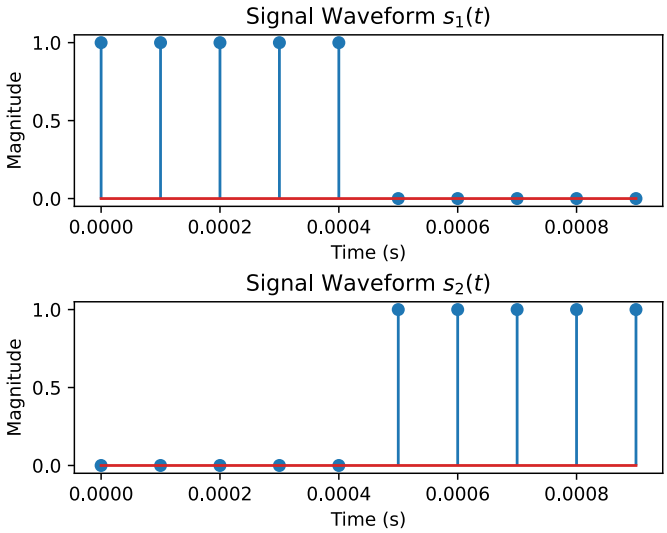


Fig. 3: Manchester encoding representations of a high symbol ($s_1(t)$) and a low symbol ($s_2(t)$)

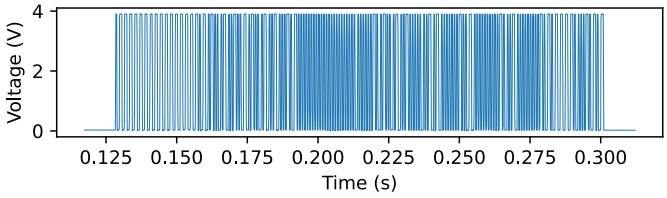


Fig. 4: Transmission burst received from the digital output of the RXB6

b) *Packet format:* The transmission from the thermostat comes in three bursts, one of which is presented in Fig. 4. Each burst follows the structure shown in Fig. 5. The 11 fields can be split up into 2 regions, a preamble that remains constant across all transmissions and every thermostat, and the data region.

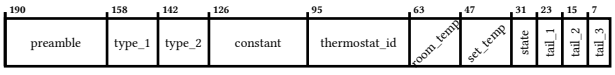


Fig. 5: Data layout over the air

The preamble fields are preamble, type_{1,2}, and constant. Their names reflect an observation of their structure. The preamble field consists of the sequence 10 repeated across 32 bits (0xAAAAAAAA). The type_{1,2} fields each contain the same bit pattern of 1101001110010001 (0xD391). The constant field contains the value 0x0DFFFFFE. All 96 bits across these fields has been determined to be the preamble. The function of the first pattern may be to indicate the beginning of a frame, the middle repeated sequence may be distinct to this specific packet type and device type, allowing receivers to differentiate the packet types over the air, and the last constant section may be a start of data delimiter with many edges to ensure manchester clock synchronization.

The data carrying fields are thermostat_id, room_temp, set_temp, state, and tail_{1,2,3}. The labels for these fields are based off of the observed function of each field, as the packet format is not documented publicly. The thermostat_id field was observed to be a unique identifier per thermostat. The room_temp and set_temp fields contain the respective values from the thermostat in units of deci-degrees ($\frac{1}{10}$th of a degree celsius). The state field was observed as a constant 0x00 value for all transmissions received except for transmissions sent during the power-on of the thermostat where the value was observed as 0x64.

The fields tail_{1,2,3} are believed to make up an error correcting code up to 24 bits long. The bit pattern is consistent across transmissions with the same data, and not clearly correlated across transmissions with slight changes in their data fields. This observation suggests that this may be an error correcting code, or integrity hash of some sort. In the process of understanding the packet format, many CRC algorithms were tested against the data fields and the tail bytes with no positive match.

For the implementation, error correction was not implemented, the preamble detection proved strong enough to reject invalid data. To be more compatible however, it would be wise to determine the algorithm used, and implement a check on reception.

B. Reception

a) *Frontend receiver:* As 433MHz is a common free-use and hobby frequency band, there are many inexpensive receivers on the market. As the transmitter used is relatively inexpensive, low cost OOK receivers were selected to act as the front-end. The implementation discussed will work with the digital, rail-to-rail, demodulated signal produced by these receivers. The receivers initially selected are listed in Table I. As the RXB6 and XD-RF-5V are the lowest cost options, they became the main focus of later tests and implementations. As will be discussed in the Discussion section, the RXB6 provided the best input for the ASIC implementation.

Table I: Low-cost OOK receivers

Device	Cost
Aurel RX-4MM5-F	8.0€
Aurel AC-RX2_CS	6.5€
HopeRF RFM210LCF-A	3€
HopeRF RFM210LCF	3€
RXB6	2.5€
XD-RF-5V	1.25€[1]

[1]This receiver comes in a Tx-Rx pair. The cost shown is half of the 2.5€ total cost for the pair.

b) *Cocotb test bench:* In order to ensure that a verilog design function as expected extensive testing is required. It would cost a lot of time and money to submit an incorrect design to a foundry as these silicon chips take around 5 months to be produced and received. [9] The tools that Tiny Tapeout suggests, and the tools used in this project are Icarus Verilog for verilog simulation, cocotb for driving and validating the outputs of the device under test or DUT, and GTKWave to view the output waveform for further analysis. Each of these tools are free and open source.

The cocotb test-bench for my design loads a CSV file containing real-life captured digital signals from the transmitter. The digital samples are then fed into the verilog code at 10kHz which matches the sample rate from the captured waveform. This provides a very reproducible and realistic decoding scenario for the decoder.

As each transmission contains thousands of samples, catching problems with the implementation by hand becomes very tedious and error-prone. To avoid this manual process, the test-bench also contains a known valid decoding of each transmission burst as well as the start and end timing of each burst. The test bench validates the decoded message against the known valid message at every step of the decoding process. This allows discrepancies to be caught as soon as they happen.

III. Results

A. Manchester decoding

A main benefit of manchester coding is the inclusion of a clock signal. To recover the clock signal and therefore the perfect sampling positions, a phase-locked-loop (PLL) can be utilized. As a PLL requires voltage controlled oscillators, it is not possible to implement such a system with the purely digital logic available. An alternative method of oversampling in order to detect clock edges has been implemented [10]. This method requires a higher sampling frequency than the ideal frequency of $F_s = 2F_b$ where F_b represents the bit frequency. Since the transmission bit frequency is in the order of 1kHz, oversampling at 20kHz was chosen. This sampling frequency will be generated by a co-processor microcontroller on the Tiny Tapeout motherboard.

Due to the shape of manchester encoding symbols as shown in Fig. 3, there exists 2 bit combinations that produce an extra transition that does not carry data, only existing as a transition between the start and end of each symbol. The bit patterns 11 or 00 result in 3 transitions, two of which contain the data. The system as shown in Fig. 2 makes use of the state machine in Fig. 6 to differentiate between the edges carrying data and the extraneous edges.

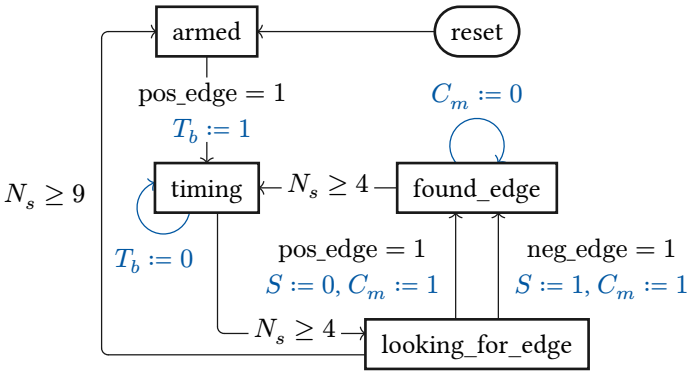


Fig. 6: Manchester decoding state machine

As the bit time has become well known through testing and has not been found to deviate during a transmission, a timer based approach was implemented, shown in Fig. 6. The period of the manchester symbols was determined to be $N_m = 18$ samples long. The state machine contains 3 states with an edge triggered by a timer. N_s represents the value stored in a 4 bit timer that is reset upon any change of state.

The blue text on edge transitions are output values, where black text on edge transitions represent the edge's condition. The 3 outputs are: S representing the current symbol, C_m representing the manchester clock, and T_b representing transmission_begin. S and C_m are used as the input bit and clock for the data shift register. T_b is connected to the asynchronous reset of the shift register.

The state machine starts in the armed state, this state waits for the first observed positive edge on the digital_in line. Upon leaving the armed state, the T_b signal is set high, signalling to serial decode module that a new transmission is expected. The edge detection behavior was chosen as the signal captures as seen in Fig. 4 start with a long period of low bits from the receiver followed by a rising edge at the start of the first manchester symbol. The reason for the receiver's behavior is discussed later in Section IV.A.

Once the state machine has entered the timing state the machine will wait 5 clock cycles — until $N_s \geq 4$. This delay ensures that the machine will ignore the rising or falling edges between two manchester encoding symbols.

In the looking_for_edge state, there are 3 possible edges that the state machine can follow. Either the state machine reaches a time-out after 10 clock cycles — $N \geq 9$ or half of the symbol period — where no manchester encoding edge has been detected due to the extended period of zeros following a transmission burst. Alternatively, on any transition of the digital_in signal, the state will transition to found_edge, with the input bit S set according to the manchester symbol observed. The manchester clock C_m is also toggled upon any edge.

found_edge will then transition back to timing after a period of 5 clock cycles or an approximation of half of the symbol period. This delay, combined with the delay from the timing state ensures that the looking_for_edge state is entered before the next data-carrying edge, but after the edge of two manchester encoding symbols.

The behavior of the state machine through all of these states has been validated using a cocotb test bench covered in Section II.B.b.

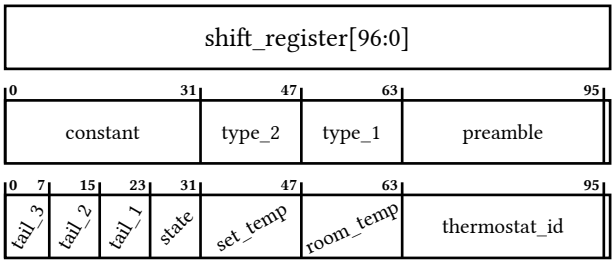


Fig. 7: Data storage layout inside of the state machine.[2]

B. *Verilog design*

As the preamble does not need to be stored, only validated in flight, the design only requires an internal shift register of 97 bits as shown in Fig. 7. The register is first used as a sliding window into the last 95 bits received, corelating them to the known preamble values. Once a match has occurred, the shift register will clear with a value of 97'b1, i.e., 96 bits of 0 with the least significant bit set to a 1. The shift register is then loaded with the next 96 bits received. Once 96 bits have been received, the initial set bit will fall into the 97th position, signifying that the register is full, and halting the reception of new bits. This optimization significantly cut down on the logic density in the tile. Since this optimization was implemented late in the development, the space savings were not able to be utilized with more logic, as the design had a strict submission date.

To interoperate with other devices, the chip contains a simple address based 8-bit wide parallel interface. The interface was initially designed when the space on the chip was limited. Had there been more time left over for this interface, it may have instead been implemented with a standard chip-to-chip interface such as I2C or SPI.

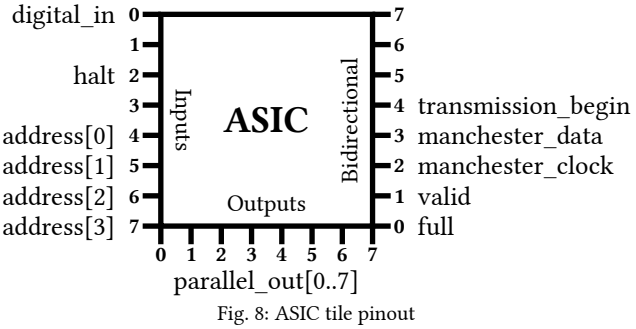


Fig. 8: ASIC tile pinout

C. *Tiny Tapeout Submission*

[2]Bit indexes represent the position in a shift register. Bit 0 being the most recently received.

Once the design has been thoroughly tested using the cocotb test benches, the design was submitted to Tiny Tapeout. As the development of the design was done using the provided template, submission and validation was simple. The template repository contains predefined workflows that will build, test in simulation, harden, run DRC, and run gate level tests automatically for every commit. This allows designers to know that if these tests pass, their design is ready for submission. These workflows also generate 2D and 3D visualizations of the ASIC post-implementation. The final design as it was submitted is shown in Fig. 9.

The main design changes to prepare an existing verilog module to be incorporated into a Tiny Tapeout tile is connecting to the given inputs and outputs. Tiny Tapeout provides 8 output pins, 8 input pins, and 8 bi-directional pins for each tile. These pins are multiplexed between the designs when they are selected. The pinout for the design in this paper is shown in Fig. 8.

The end of this BEP aligned with the closing date of the tt07[3] shuttle. The decoder as it was submitted to Tiny Tapeout is available on both GitHub[4] and the website of TinyTapeout[5].

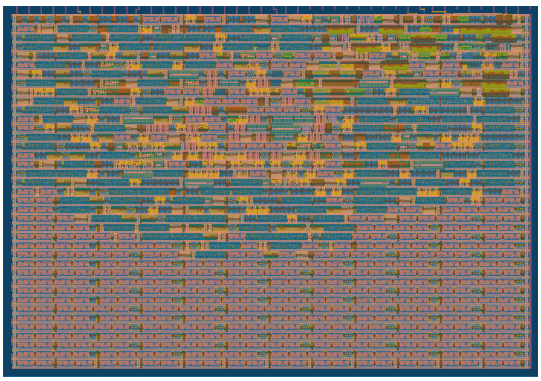


Fig. 9: Submitted ASIC

IV. Discussion

A. Compatibility

Although there were many selected receivers, the first iteration of the decoder was designed around digital waveform captures from the RXB6. This lead the design to heavily lean into the design choices of the RXB6 itself, losing compatibility with at least one other receiver. All receivers were unable to be tested for. The only known compatible receiver is the RXB6 as analyzed previously. The XD-RF-5V was also used in captures along side the RXB6, making it possible to easily test its received waveforms. The state machine will begin to decode the incoming transmission, but it will not start decoding at the exact start of the transmission.

The major difference in the waveforms captured by either receiver is how they handle the periods between the data bursts. The RXB6 will output a sequence of low bits for the whole duration, where as the XD-RF-5V will start with a period of low bits, followed by digital noise before the next transmission burst. The low bits from the RXB6 receiver between pulses lead to the RXB6 receiver being very compatible with the implementation chosen, as there were very well delineated message bursts.

A possible explanation for the difference in behavior of these two receivers could be their different implementations of automatic gain control (AGC). The observed low bits from the RXB6 imply that the AGC has not raised the gain enough for the background noise to be received as digital noise. The observed period of zero bits after the transmission burst on the XD-RF-5V may also be due to the AGC slowly raising the gain, until the gain is high enough to result in the digital noise observed.

Future work should focus on expanding compatibility with simpler or differently designed receivers. The implementation may not need to change substantially, as the manchester decoding implementation should be the same. The main area that would need to be changed is the preamble validation step — allowing for missing or mangled starts of the preamble — as well as the removal of the armed state — which is only compatible with the RXB6 or other similar receivers.

B. Hardening

When testing verilog code after synthesis, Tiny Tapeout recommends and provides tooling for gate-level testing. Gate level testing simulates the design as it was synthesized, i.e, simulating the behavior of each transistor. Once the design has been synthesized, much information regarding the wiring, i.e., the netlist, has been lost to optimizations. This limits the gate-level test to only be able to interact with the chip – both setting registers and reading wires – via the external pins.

To allow for as much introspection as possible into the workings of my design, i chose to expose many internal signals on the output pins, as well as an 8 bit wide parallel bus, allowing the test-bench or a micro-controller to read out the state of the chip.

When hardening designs, clock timing is very important. Design and routing tools will attempt to satisfy timing constraints, however certain verilog code can make those constraints harder to satisfy. A rule of thumb picked up from my work on this design has been to leave clock signals untouched, utilizing them for clocking of always blocks only. My initial attempt to gate the clock to the state machine with the halt pin was implemented by passing a signal of the original clock and-ed with the halt signal to the clock line of the state ma-

[3]https://app.tinytapeout.com/shuttles/tt07

[4]https://github.com/DusterTheFirst/tt07-bep-decode/tree/ae403d4944ff634e624b8e098ec300b116aa458b

[5]https://app.tinytapeout.com/projects/725

chine. This change resulted in "hold violations in the design at the Typical corner". These violations suggest some issue with timings. The correct way to implement a gate like this would be to add an enable pin to the module, checking the value of the enable pin with an if statement in the clocked state machine always block. This change removed the violations from my design.

When designing hardware around a clock signal, an important consideration are clock domains. Ensure that each clocked always block that will interact with each-other, either directly or indirectly, are using the same wire or signal as their clock. [11] In practice, that means that generated, lower frequency clock lines — like the manchester clock — should be implemented with logic inside of an always block, clocked with the same global clock as all other modules.

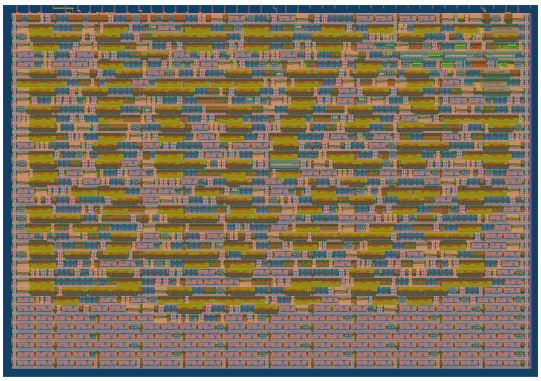


Fig. 10: First working implementation

C. *ASIC optimizations*

With Tiny Tapeout, the size 160x100µm area can be restrictive. It is possible to expand the tile area up to 8x2 tiles (1280x200µm), however for this project it was decided that space efficiency was an important consideration.

When trying to lower the density/utilization of a design, it can be hard to tell which specific module or line of code is causing what utilization. There are simple heuristics that have been taken into account in order to decrease the utilization from 62% (Fig. 10) to the final 32% (Fig. 9). Registers and other flip-flop based logic tends to be large components.

"In an FPGA, registers are 'free' in that every LUT has a flop. So people are used to doing lots of pipelining and registering. But in an ASIC, these are the most 'expensive' cells in terms of area, so it's better to do more with combinational and only use flops where necessary. This might mean doing deeper levels of combinational logic with fewer flops in between." [12]

Initial designs used a 194 bit shift register, which has been halved in length as discussed earlier.

Asynchronous reset implementations for large banks of registers reduces the required amount of logic cells. The tradeoff of an asynchronous reset is that the reset operation is no longer synchronized with the clock signal. In the case of the manchester decoder strict clock synchronization will not need to be achieved.

Using negative edge resets plays into the design of the logic cells in most PDKs, including the utilized SkyWater 130 PDK. Flip flop implementations tend to implement a negative edge reset as they are simpler to implement with transistors. If a positive edge reset is utilized, an inverter will be placed, possibly multiple times for the one reset line.

V. Conclusions

The design initially took a more complicated approach to its implementation which left much room for optimizations and reductions, drastically simplifying the state machine and the shift register implementations. This may be equally applicable to other designs as space constraints are more limiting in ASIC design.

Tiny Tapeout provided a very structured and low cost approach to ASIC development. Combined with a supportive community, Tiny Tapeout enables designers with little experience with ASIC design to bring their designs to reality.

The development of the receiver implementation was the main time factor, once the design was finished in simulation, the next steps were very straight forward with the main roadblocks being in Section IV.B and Section IV.C,

As discussed in Section IV.A, the design in this paper was heavily developed around the output of the RXB6 receiver. Future work should test the receiver implementation against waveforms captured from other receivers, and correct for design decisions that have caused the implementation to lose compatibility. Two aspects that require focus are decreasing the strictness of the preamble validation step, and avoiding the need for the armed state. Compatibility with a wider range of receivers should be the goal of future work on this design.

References

[1] T. Ansell and M. Saligane, "The missing pieces of open design enablement: a recent history of google efforts," in *Proceedings of the 39th International Conference on Computer-Aided Design*, in ICCAD '20. Virtual Event, USA: Association for Computing Machinery, 2020. doi: 10.1145/3400302.3415736.

[2] Google and Skywater Technologies, "Google + Skywater Technology FOSS 130nm Production PDK." [Online]. Available: https://github.com/google/skywater-pdk

[3] YosysHQ, "OSS CAD Suite." [Online]. Available: https://github.com/YosysHQ/oss-cad-suite-build

[4] M. Venn and U. Shaked, "Tiny Tapeout." [Online]. Available: https://tinytapeout.com/

[5] Efabless, "Caravel Harness." [Online]. Available: https://github.com/efabless/caravel

[6] M. Venn and U. Shaked, "Tiny Tapeout 3." [Online]. Available: https://tinytapeout.com/runs/tt03/

[7] M. Shalan and T. Edwards, "Building OpenLANE: A 130nm OpenROAD-based Tapeout- Proven Flow : Invited Paper," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–6.

[8] F. M. Willems and A. Alvarado, "COMMUNICATION THEORY (5ETB0) Course Reader," 2023.

[9] M. Venn and U. Shaked, "Tiny Tapeout Chips." [Online]. Available: https://tinytapeout.com/runs/

[10] J. Jimenez, J. Martin, A. Astarloa, and A. Zuloaga, "Manchester decoding algorithm for multifunction vehicle bus," in *2004 IEEE International Conference on Industrial Technology, 2004. IEEE ICIT '04.*, 2004, pp. 769–774. doi: 10.1109/ICIT.2004.1490172.

[11] L. Kleeman and A. Cantoni, "Metastable Behavior in Digital Systems," *IEEE Design & Test of Computers*, vol. 4, no. 6, pp. 4–19, 1987, doi: 10.1109/MDT.1987.295189.

[12] M. Venn and U. Shaked, "FPGA to ASIC." [Online]. Available: https://tinytapeout.com/hdl/fpga_vs_asic/